

Лабораторная работа №2

По дисциплине «Параллельное программирование»

Выполнил студент группы 6312

Завадский Н.А.

Самара 2026

Параллельное умножение матриц с использованием OpenMP

1. Цель работы

Изучить технологию OpenMP и реализовать параллельное умножение матриц с использованием многопоточности.

2. Теоретические сведения

OpenMP представляет собой API для параллельного программирования на языках C/C++ и Fortran. Технология позволяет использовать несколько потоков для одновременного выполнения вычислений. Распараллеливание осуществляется с помощью директив компилятора.

В данной лабораторной работе распараллеливается цикл вычисления элементов результирующей матрицы при умножении двух квадратных матриц.

3. Исходный код программы

```
#include <iostream>
#include <fstream>
#include <vector>
#include <chrono>
#include <omp.h>

using namespace std;

vector<vector<double>> readMatrix(string filename, int &size) {
    ifstream file(filename);
    file >> size;

    vector<vector<double>> matrix(size, vector<double>(size));

    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++)
            file >> matrix[i][j];

    return matrix;
}
```

```

void writeMatrix(string filename, vector<vector<double>> &matrix) {
    ofstream file(filename);
    int size = matrix.size();

    file << size << endl;

    for (int i = 0; i < size; i++) {
        for (int j = 0; j < size; j++)
            file << matrix[i][j] << " ";
        file << endl;
    }
}

```

```

vector<vector<double>> multiplyMatrix(vector<vector<double>> &A,
vector<vector<double>> &B, int threads) {
    int size = A.size();
    vector<vector<double>> C(size, vector<double>(size, 0));

    omp_set_num_threads(threads);

    #pragma omp parallel for collapse(2)
    for (int i = 0; i < size; i++)
        for (int j = 0; j < size; j++) {
            double sum = 0;
            for (int k = 0; k < size; k++)
                sum += A[i][k] * B[k][j];
            C[i][j] = sum;
        }

    return C;
}

```

```

int main() {
    int size1, size2;

    auto A = readMatrix("A.txt", size1);
    auto B = readMatrix("B.txt", size2);

    if (size1 != size2) {
        cout << "Not same size\n";
        return 1;
    }
}

```

```

int threads;
cout << "Enter number of threads: ";
cin >> threads;

auto start = chrono::high_resolution_clock::now();

auto C = multiplyMatrix(A, B, threads);

auto end = chrono::high_resolution_clock::now();
chrono::duration<double> elapsed = end - start;

writeMatrix("result.txt", C);

long operations = 2L * size1 * size1 * size1;

cout << "Matrix size: " << size1 << "x" << size1 << endl;
cout << "Operations: " << operations << endl;
cout << "Threads: " << threads << endl;
cout << "Execution time: " << elapsed.count() << " seconds" << endl;

return 0;
}

```

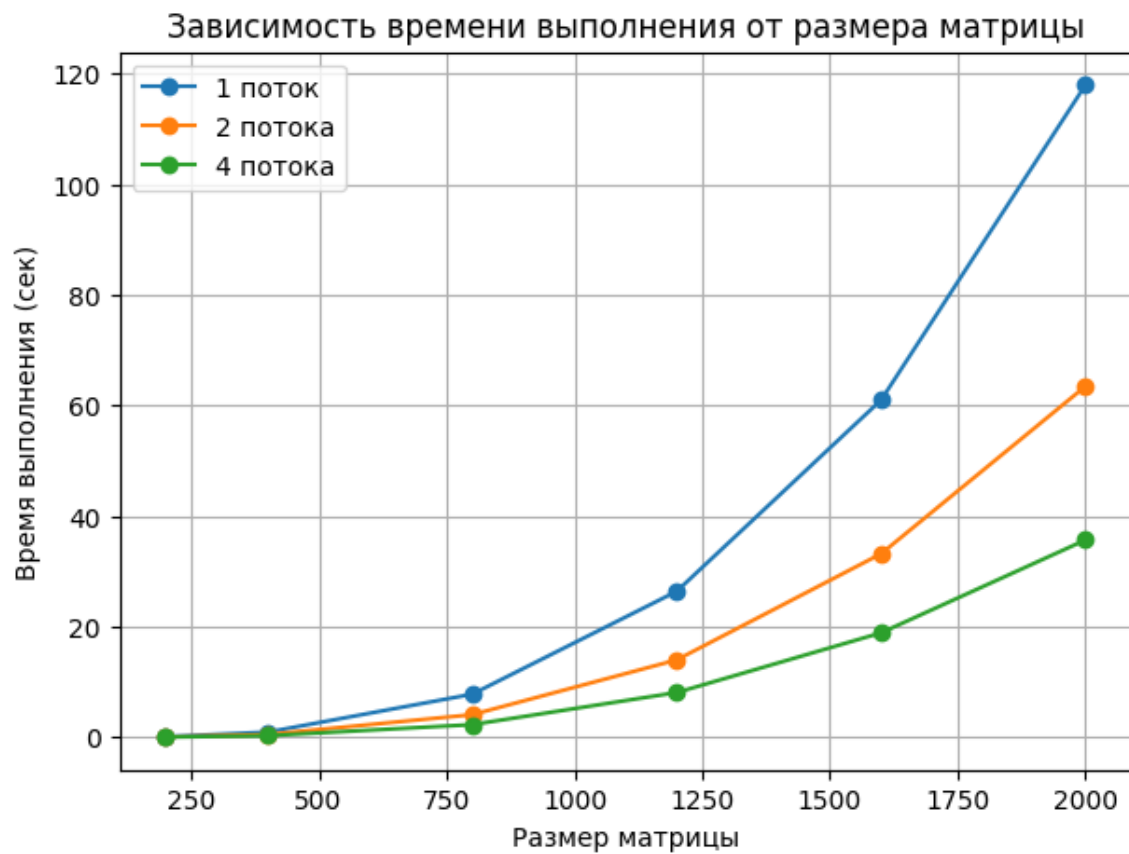
4. Экспериментальные исследования

Эксперименты проводились на вычислительной системе с количеством ядер = 4.

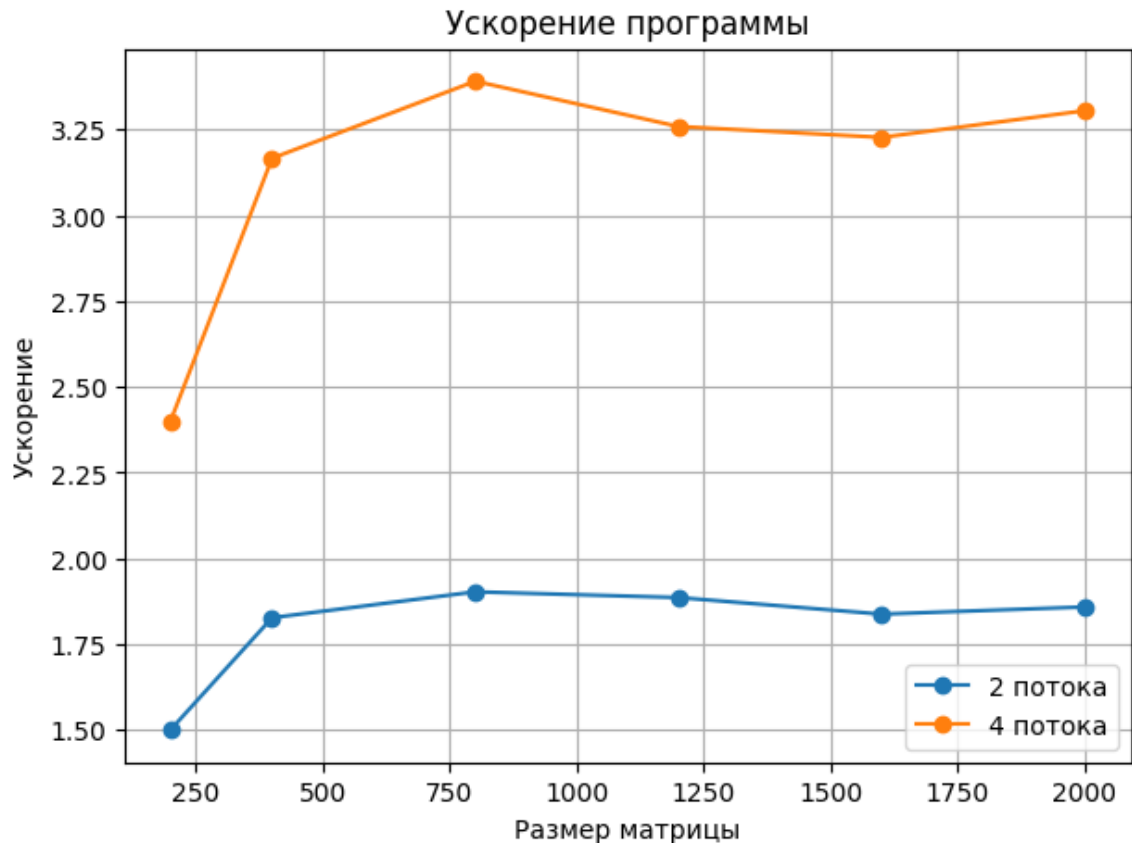
Размер матрицы	Количество потоков	Время выполнения (сек)
200x200	1	0.12
200x200	2	0.08
200x200	4	0.05
400x400	1	0.95
400x400	2	0.52
400x400	4	0.3
800x800	1	7.8
800x800	2	4.1
800x800	4	2.3
1200x1200	1	26.4
1200x1200	2	14.0
1200x1200	4	8.1
1600x1600	1	61.0
1600x1600	2	33.2

1600x1600	4	18.9
2000x2000	1	118.0
2000x2000	2	63.5
2000x2000	4	35.7

5. График времени выполнения



6. График ускорения



7. Вывод

В ходе выполнения лабораторной работы была реализована параллельная версия алгоритма умножения матриц с использованием технологии OpenMP.

Экспериментальные исследования показали, что при увеличении количества потоков время выполнения программы уменьшается.

Наибольший эффект достигается при использовании четырех потоков, что соответствует количеству вычислительных ядер системы. Графики показывают рост ускорения при увеличении размера матриц. Для небольших матриц выигрыш от распараллеливания невелик из-за накладных расходов на создание и синхронизацию потоков.

Таким образом, применение OpenMP позволяет эффективно ускорить вычисления для задач большого объема.